

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE CODE: CSA1401

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 3

Total Marks:50

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I E. Gowtham ¹⁹²³¹¹¹⁹⁰ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: E. Gowtham

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation, including lexical, syntax, semantic, logical, and runtime errors.**

Given Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
    int i, sum = 0;
    for(i = 0; i <= n; i++) {
        sum = sum + arr[i];
    }
    return sum;
}

int main() {
    int arr[5] = {1,2,3,4,5}
    int total;

    total = sumArray(arr);

    if(total == 15) {
        printf("Correct Sum\n")
    }

    int a = 10, b = 0;
    int c = a / b;

    int *p;
    *p = 20;

    return 0;
}
```

Tasks:

- (a) Error Identification (Analysis Level)
 1. Identify all the errors present in the program. For each error:
 2. Specify the line number
 3. Clearly describe the nature of the error
- (b) Classification of Errors (Understanding + Analysis)

1. Classify each identified error into the appropriate compiler phase:
2. Lexical Errors
3. Syntax Errors
4. Semantic Errors
5. Logical Errors
6. Runtime Errors

(c) Compilation Behavior (Evaluation Level)

1. Analyze which errors will be detected during compilation and which will only appear during execution.
2. Explain how early-stage errors (lexical/syntax) may prevent detection of later-stage errors.

(d) Code Correction (Application Level)

1. Rewrite the corrected version of the program such that:
2. All errors are removed
3. The program executes correctly
4. Proper syntax and semantics are maintained

(e) Impact Analysis (Evaluation Level)

1. Discuss the impact of at least three errors on program execution.
2. Explain how incorrect constructs like improper pointer usage, division by zero, and infinite loops affect system behavior.

(f) Advanced Compiler Perspective (Create Level)

1. Suggest how a compiler can be improved to:
2. Provide better error messages
3. Perform early detection of runtime errors
4. Propose at least two enhancements in compiler design for efficient debugging.

Q2. The following C program contains **multiple** lexical errors introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the lexical analysis phase of a compiler. Carefully examine the program and answer the questions given below.

```
#include <stdio.h>

int main() {
    int lnum = 100;
    float rate@ = 25.5;
    char ch = 'abc';
    int total = 50$20;
    int num#1 = 30;
    int %count = 40;
```

```
    printf("Welcome to Compiler Design);
```

```
    int x = 09;
    int y = 0xZ12;
    int z = 12.3.4;
```



```

char str1[] = "Hello;
char str2[] = "World";

int _valid = 10;
int invalid-id = 20;

float value = 5..6;
int data = 10@5;

char c = 'xy';
int 3value = 60;

printf("Value: %d\n", total);

return 0;
}

```

Tasks

(a) Error Identification

- i) Identify all lexical errors in the program.
- ii) List the line number and the invalid token.

(b) Error Classification

- i) For each error, explain why it is considered a lexical error.
- ii) Specify which rule of token formation is violated:
- iii) Identifier rules
- iv) Constant formation
- v) String/character literal rules
- vi) Operator/symbol rules

(c) Token Correction

- i) Rewrite each invalid token into a valid token.
- ii) Example: 2value → value2

(d) Compiler Behavior Analysis

- i) Explain how a lexical analyzer (scanner) processes such errors.
- ii) Discuss whether the compiler:
- iii) Stops immediately
- iv) Continues scanning using error recovery

(e) Advanced Thinking

- i) Design a simple rule (or DFA idea) to detect invalid identifiers starting with digits.
- ii) Suggest how error messages can be improved for beginners.

Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

- A. Lexical Errors
- B. Syntax Errors
- C. Semantic Errors
- D. Intermediate Code Generation Issues (logic/precedence)
- E. Code Optimization Issues (redundancy/inefficiency)
- F. Runtime / Target Code Errors

Carefully analyze the program and answer the questions below.


```

#include <stdio.h>

int divide(int a, int b) {
    return a / b
}

int main() {
    int #value = 20;
    float num = 15.5
    int result;

    result = divide(10);

    if(result = 5) {
        printf("Result is 5\n")
    }

    int x = 10;
    int y = 0;
    int z = x / y;

    int arr[4] = {1,2,3,4};
    printf("%d\n", arr[4]);

    int *ptr;
    *ptr = 25;

    int total;
    total = x + y * result + 0;

    while(x < 20) {
        printf("%d\n", x);
    }

    return 0;
}

```

Do the following Tasks

(a) Identify Errors (Analysis Level)

- I. Identify all errors in the program.
- II. Mention line number + error description.

(b) Classify by Compiler Phase

Classify each error into:

- I. Lexical (invalid symbols like @num)
- II. Syntax (missing semicolon, brackets)
- III. Semantic (function argument mismatch)
- IV. Intermediate Code Issues (wrong expression handling/precedence)
- V. Optimization Issues (redundant + 0)
- VI. Runtime Errors (division by zero, invalid pointer access)

(c) **Compilation vs Execution (Evaluation)**

- I. Which errors are detected at **compile time**?
- II. Which errors appear only during **execution**?
- III. Explain why some errors prevent further compilation.

(d) **Correct the Program (Application)**

- I. Rewrite the **fully corrected** program.
- II. Ensure:
 - A. Proper syntax
 - B. Correct semantics
 - C. Safe execution

(e) **Deep Analysis (Evaluation)**

- I. Explain how **incorrect pointer usage** affects runtime.
- II. Analyze how **array bounds violation** impacts memory.
- III. Discuss how **operator precedence errors** affect intermediate code.

(f) **Advanced Task (Create Level)**

- I. Suggest **2 improvements** in compiler design to:
 - A. Detect runtime errors earlier
 - B. Improve debugging messages
- II. Propose how **optimization phase** can remove redundant expressions.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided

Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

1) a) Error Identification

Line No.	Incorrect code	Error Type	Description
28	for (i=0; i <= n; i++)	Logical	Loop runs one extra time
34	int arr[5] = {1, 2, 3, 4, 5}	Syntax	Missing ;
36	total = sumArray(arr);	Semantic	It required two arguments
38	printf("Correct in")	Syntax	Missing ;
41	int c = a / b;	Run time	Divide by zero
42-43	int *p; *p = 20;	Runtime	Pointer is uninitialized

b) Classification of Errors

i) Error: Missing semicolon after any array declaration

Compiler phase: Syntax Analysis

Reason: Violates C grammar rules.

ii) Error: Missing function argument

Compiler phase: Semantic Analysis

Reason: Function call does not match its declaration

iii) Error: Missing semicolon after printf

Compiler phase: Syntax Analysis

Reason: parser expects a semicolon after every statement

iv) Error: $i \leq n$ loop

Compiler phase: logical Error

Reason: Accesses one element beyond the array

v) Error: Divide by zero

Compiler phase: Runtime Error

Reason: Can't be detected by syntax analysis

vi) Error: uninitialized pointer

Compiler phase: Runtime Error

Reason: unknown memory location

c) Compilation Behavior

Error detected during (semi) compilation

Missing semicolon after array declaration

Missing semicolon after printf

Missing incorrect no. of arguments in sumArray

Error detected during execution

Division by zero

Invalid pointer difference

Array index out of bounds due to $i \leq n$

How early-stage errors affect compilation

- The compiler first performs lexical and syntax analysis
- If syntax errors exist, parsing stops
- Semantic analysis and code generation may not proceed until syntax errors are corrected

d) Corrected Program

```
#include <stdio.h>
```

```
int SumArray(int arr[], int n) {
```

```
    int i, sum = 0;
```

```
    for (i = 0; i < n; i++) {
```

```
        sum += arr[i];
```

```
    } return sum;
```

```
}
```

```
int main() {
```

```
    int arr[5] = {1, 2, 3, 4, 5};
```

```
    int total;
```

```
    total = SumArray(arr, 5);
```

```
    if (total == 15) {
```

```
        printf("Corrected Sum\n");
```

```
        int a = 10;
```

```
        int b = 2;
```

```
        int c = a/b;
```

```
        int value = 20;
```



```
int *p = &value;
```

```
*p = 20;
```

```
printf("Division Result = %.d\n", c);
```

```
printf("pointer value = %.d\n", *p);
```

```
return 0;
```

```
}
```

e) Impact Analysis

1. Improper pointer usage

Error

```
int *p
```

```
*p = 20;
```

Impact:

- pointer does not point to valid memory
- Causes segmentation fault
- Program terminates unexpectedly

2. Division by zero

Error

```
int c = a/b; // where b = 0
```

Impact:

- causes arithmetic exception
- Program crashes during execution
- produces undefined behavior.

3. Incorrect loop conditions

```
for (i = 0; i <= n; i++)
```


Impact

- Accesses `arr[n]`, which is outside the array
- Reads invalid memory
- May produce incorrect output

f) Advanced compiler perspective

1. Better error messages

1. Better Error Messages

- Displaying the exact line number and highlighting the incorrect token.
- Suggesting possible fixes.

2. Error detection of Runtime Errors.

- Warn about possible division by zero
- Detect uninitialized pointer
- Warn about possible array index overflow.
- Perform static analysis before execution

3. Two compiler Design Enhancements

i) Static Analysis

ii) Intelligent Debugging

Suggest automatic fixes

Detect unreachable codes

check division by zero

2) a) Error Identification

Line No.	Invalid Token
84	lnum
85	rate@
86	'abc'
87	\$
88	num#1
89	%.count
90	"Welcome to compiler Design
91	09
92	0xZ12
93	12.3.4
95	"Hello
98	invalid -id
99	5..6
100	10@5
101	'xy'
102	3value

b) Error Classification

Invalid Token	Lexical Rule
lnum	Identifier rules
rate@	Identifier rules
'abc'	Character literal
\$	Symbol rules

num#1

%count

missing "

09

0xZ12

12.3.4

"Hello

invalid-id

5..6

10@5

'xy'

3value

c)Token Correction

Invalid Token

num

rate@

'abc'

50\$ 20

num#1

%count

"Welcome to compiler

Design n"

Identifier rules

Identifier rules

Constant

Constant

Constant

Constant

String literal rules

Identifier rules

constant

symbol rules

character literal

Identifier rules

Correct Token

num1

rate

'a'

50+20

num1

count

"Welcome to compiler

Design n"

0x212

12.3.4

"Hello"

invalid-id

5..6

10@5

'xy'

3value

0x12

12.34

"Hello"

invalid_id

5.6

10+5

'x b l l'

value3

1) Compiler Behavior Analysis

i) Lexical Analyzer:

- Groups characters into tokens
- Reports invalid tokens as lexical errors
- Passes valid tokens to the parser
- Compares each token with C language

ii) Does the compiler stop immediately?

No, modern compiler using usually perform error recovery.

They:

- Skip the invalid token
- Report the current lexical error
- Continue scanning the remaining source code
- Report multiple errors in a single compilation.

e) Advanced Thinking

is DFA idea to detect Invalid Identifiers

Rule

An Identifier:

- must begin with a letter (A-Z) (a-z) or (-)
- Remaining characters may be letters, digits or underscore

Start

↓

letter / _ → Accept state

↓

Digit → Reject state

- If the first character is a digit, the scanner immediately rejects the token as an invalid identifier

ii) Improved Error messages for Beginners

line 84:

Identifier 'inum' cannot begin with digit.

Suggestion:

Rename it as num or value

These descriptive messages makes it easier for beginners to understand and correct errors.

3. a) Identify Error

Line No	Incorrect code	Error Type
142	return a b	Syntax
145	int #value = 20;	Lexical
146	Float num = 15.5;	Syntax
148	result = divide(10);	Semantic
149	if (result = 5)	Logical
150	printf("Result")	Syntax
154	int z = x/y	Runtime
156	printf("%.d\n", arr[43]);	Runtime
157-158	int *ptr; *ptr = 25;	Runtime
160	total = x + y * result + 0;	Optimization
161-163	while (x < 20)	Logical

b) Classify Compiler phase

i) Error: #value

Compiler phase: Lexical

Reason: Invalid identifier due to #

ii) Error: + 0

Compiler phase: Optimization

Reason: Redundant expression that has no effect.

iii) Error: Infinite loop

Compiler phase: logical

Reason: loop condition never changes

c) Compilation vs Execution

- Invalid Identifier
- missing semicolons
- Division by zero
- Infinite loop
- Array index out of bounds
- Division by zero

d) Corrected program

```
#include <stdio.h>
```

```
int divide (int a, int b) {  
    return a/b;
```

```
}
```

```
int main() {
```

```
    int value = 20;
```

```
    float num = 15.5;
```

```
    int result;
```

```
    result = divide(10, 2);
```

```
    if (result == 5) {
```

```
        printf("Result is 5\n");
```

```
    }
```



```

int x=10;
int y=2;
int z=x/y;
int arr[4]= {1,2,3,4};
printf("%d\n", arr[3]);
int data=25;
int *ptr = &data;
*ptr = 25;
int total;
total = x+y * result;
while (x<20) {
    printf("%d\n", x);
    x++;
}
printf(z);
printf(*ptr);
printf(total);
return 0;
}

```

e) effect of Incorrect pointer usage

```

int *ptr
*ptr = 25;
printf(arr[4]);
if (result = 5)

```

- often results in a segmentation fault
- may print garbage values.

- assigns the value 5 to result.
- The condition always evaluates as true.

f) Advanced Task

- i) Two improvements in Compiler Design
 - 1) Early detection of Runtime errors
 - 2) Improved Debugging Messages
- ii) Optimization phase
 - Remove unnecessary operations
 - produces more efficient intermediate and target code.
 - Improves execution speed and reduces instruction count.

total = x + y * result;